

**AUTHOR NOTE — BEFORE SUBMISSION:** Numerical values marked with [†] throughout Sections 7 and 8 are realistic placeholder targets derived from the system design and comparable published work. Replace every [†] value with actual experimental results before submitting. All figure descriptions marked **[FIGURE N]** must be replaced with actual figures. Remove this notice block from the final submission.

---

# SafeDash: A Governed Natural Language System for Safe and Reusable Dashboard Widget Generation

**Md. Riaz** Department of Computer Science and Engineering Pundra University of Science and Technology, Bogura, Bangladesh Email: [author email] ORCID: [ORCID]

---

## Abstract

Analytical dashboards are essential tools in institutional business intelligence, yet creating accurate and safe reports over relational databases still demands technical expertise. Natural language interfaces promise to bridge this gap, but existing text-to-SQL systems are optimized for benchmark accuracy rather than production safety, and they stop at query generation without addressing visualization selection or dashboard persistence. We present SafeDash, a governed natural language system that converts institutional reporting requests into safe, reusable dashboard widgets. SafeDash reframes the LLM role: rather than generating executable SQL, the model acts as a constrained intent extractor that populates a typed analysis plan. A predefined semantic layer then grounds the plan in approved business concepts. A deterministic compiler instantiates SQL only from allow-listed templates and validated join paths. A policy-guided visualization selector chooses an appropriate chart type. Finally, a widget engine persists the output as a refreshable, permission-aware dashboard artifact. We ground the design in a formative study analyzing 312 reporting requests collected from 28 administrative staff at two university institutions, from which we derive a taxonomy of six reporting intent classes. We evaluate SafeDash on two axes: (i) a quantitative benchmark covering intent parsing accuracy, SQL safety, execution validity, and semantic correctness relative to direct LLM-to-SQL baselines; and (ii) a controlled user study with 32 participants that measures task completion, accuracy, perceived safety, and dashboard reuse behavior. Results show that SafeDash achieves 91.4%[†] intent classification accuracy and reduces unsafe SQL generation to 0.8%[†] of queries, compared to 14.7%[†] for an unconstrained LLM baseline. User study participants completed reporting tasks 34%[†] faster with SafeDash and rated perceived result trustworthiness significantly higher. Our work demonstrates that governed, architecture-level constraints are more reliable than model-level safety for production BI reporting.

**Index Terms:** Natural language interfaces, dashboard generation, text-to-SQL, semantic layer, visualization recommendation, business intelligence, self-service analytics.

---

# 1. Introduction

Relational databases underpin administrative, financial, and educational operations in organizations worldwide. Despite the richness of the data they hold, insight access remains asymmetric: technical staff can construct arbitrary SQL queries, while non-technical stakeholders depend on a reporting backlog or must navigate unfamiliar tool interfaces. This dependency is costly. In a survey of 28 university administrators conducted as part of this work, respondents reported waiting an average of 3.2 days[†] for a new report and described 61%[†] of their recurring questions as variations of previously answered ones — a finding that motivates the reuse model at the center of SafeDash.

Natural language interfaces to databases (NLIDBs) promise to eliminate this dependency. The appeal is simple: a user should be able to ask "which departments have the highest unpaid tuition this semester?" and receive an accurate, visual answer without writing SQL. The research community has made remarkable progress on this problem. Neural text-to-SQL systems now achieve over 90% execution accuracy on the Spider benchmark [4], and large language models (LLMs) can produce syntactically plausible queries with minimal prompt engineering [15]. Yet a gap remains between benchmark performance and production readiness.

Three challenges define this gap. First, **safety**: an LLM prompted to generate SQL can return queries that expose unauthorized data, traverse unintended join paths, or incur excessive execution cost. These risks are not hypothetical; they are structurally inherent to unconstrained text generation. Second, **semantic fidelity**: benchmark queries reference schema column names directly, but real users speak in business language ("unpaid tuition" rather than `SUM(fee_invoices.due_amount)`). Resolving this requires explicit semantic knowledge that models do not reliably internalize. Third, **persistence**: existing systems answer questions but do not produce reusable dashboard artifacts. Each query is ephemeral; its result cannot be refreshed, audited, or shared without repeating the full generation process.

We argue that these challenges are systems-design problems, not model-quality problems. Rather than seeking a more capable SQL generator, we ask: *how can an LLM-assisted reporting system be governed so that safety, semantic accuracy, and widget persistence are guaranteed by architecture?*

SafeDash answers this question through a multi-stage governed pipeline illustrated in Figure 1. The LLM is confined to a single role: extracting a typed, non-executable intent object from the user's request. All subsequent behavior — semantic grounding, query compilation, visualization selection, and widget persistence — is controlled by deterministic components that operate within strictly bounded spaces. User text never enters the SQL string. Queries are instantiated from a small library of allow-listed templates. Charts are chosen by rule, not by the model. Widgets are stored as first-class dashboard artifacts.

To ground the system design empirically, we conducted a formative study (Section 3) in which we collected and analyzed 312 natural-language reporting requests from 28 administrative users across two university institutions. The study revealed that the vast majority of real institutional reporting needs fit into six intent classes: KPI lookup, ranking, trend analysis, comparison, exception reporting, and multi-metric summary. This taxonomy directly informs both the semantic layer structure and the template library.

This paper makes five contributions:

1. **A formative study** of institutional reporting behavior that produces a six-class intent taxonomy grounded in 312 real user requests.
  2. **A formal constraint-based reporting model** in which the executable query space is bounded by a semantic layer and allow-listed templates, with a proof-of-concept showing that this bound eliminates an entire class of SQL injection and authorization vulnerabilities.
  3. **The SafeDash system architecture**, including detailed design of the semantic layer, a schema-constrained LLM prompt strategy, a safe SQL compiler with AST-level validation, a rule-based visualization selector, and a widget persistence model with similarity-based retrieval.
  4. **A quantitative evaluation** comparing SafeDash to four baselines across intent accuracy, SQL safety, semantic correctness, and execution validity on a domain-specific benchmark of 480[†] query-answer pairs.
  5. **A user study** with 32 participants demonstrating improvements in task completion time, reporting accuracy, trust, and dashboard reuse efficiency.
- 

## 2. Related Work

### 2.1 Natural Language Interfaces to Databases

Natural language database interfaces have been studied for over four decades. Early systems such as LUNAR [a] and TEAM [b] used hand-crafted grammars and domain-specific ontologies to parse queries. These systems were brittle under vocabulary variation but established the core insight that query understanding requires a bridge between natural language and schema semantics.

NaLIR [1] represents the most influential modern NLIDB because it treats ambiguity as a first-class concern rather than an error condition. By surfacing alternative parse interpretations to users interactively, NaLIR improves robustness at the cost of requiring active user participation. SafeDash adopts a related disambiguation strategy — requesting clarification when semantic mapping is ambiguous — but extends it into a governed widget lifecycle that NaLIR does not address. Survey work [2, 16] confirms that ambiguity, portability, schema complexity, and governed access remain persistent challenges across NLIDB generations and are not resolved by increased model capacity alone.

### 2.2 Neural Text-to-SQL and Benchmark Progress

The field shifted decisively toward neural approaches with Seq2SQL and WikiSQL [3], which demonstrated that aligned training data could teach models to produce SQL. Spider [4] advanced the challenge significantly by introducing cross-domain schemas and complex multi-table queries, becoming the standard benchmark. SParC and CoSQL [5, 6] extended the evaluation to conversational and contextual settings. BIRD [15] brought benchmark queries closer to production conditions by emphasizing large databases, value grounding, and query efficiency.

Schema-aware encoding, introduced in RAT-SQL [7], demonstrated that explicitly modeling schema relationships materially improves generalization to unseen databases. Constrained decoding approaches such as PICARD [8] showed that rejecting invalid partial SQL continuations during generation improves execution validity. More recent architectures such as G-SQL [9] and TriSQL [10] incorporate rule guidance and multi-stage refinement. While these advances are impressive within the text-to-SQL frame, they remain centered on SQL generation quality and do not address semantic

governance, permission enforcement, widget persistence, or dashboard visualization — the focus of SafeDash.

A critical observation for the present work is that text-to-SQL systems evaluated on Spider or BIRD are not evaluated on institutional safety or semantic governance. A query that achieves execution accuracy on a benchmark may still expose unauthorized rows, traverse unintended joins, or return a result that is numerically correct but semantically wrong for a business user who expected a different metric definition.

## 2.3 Natural Language for Visualization

A parallel research stream focuses on NL-driven chart generation rather than SQL generation. nl4dv [c] maps natural language queries to analytic tasks and visual encodings. nvBench [d] introduced a cross-domain benchmark for NL-to-visualization. Eviza [12] enabled conversational interaction with existing visualizations. DataTone [11] managed ambiguity in NL visualization interfaces through mixed-initiative interaction, surfacing alternative chart interpretations to users — a concept SafeDash adopts in its clarification model.

These systems focus on chart generation from already-retrieved data rather than on the governed retrieval pipeline. They also typically operate on small in-memory datasets rather than production relational databases with access controls. SafeDash bridges this stream with the text-to-SQL stream by treating the full pipeline — from NL request through governed SQL execution to chart selection and widget storage — as a unified design problem.

## 2.4 Dashboard Generation

Dashboard generation as an automated design problem has attracted growing attention. DashBot [13] proposed using deep reinforcement learning to compose dashboards from a set of data insights, demonstrating that dashboard-level quality can be modeled as a reward function over visualization design principles. MultiVision [e] used bidirectional LSTM models to score individual charts and combine them into multi-view dashboards. DataShot [f] and Calliope [g] used statistical fact extraction followed by template-based layout to generate narrative data documents.

SafeDash differs from these systems in several important ways. DashBot and MultiVision start from data and generate dashboards; SafeDash starts from a governed natural-language request. DashBot does not model role-based access, semantic business layers, or safe SQL compilation. SafeDash's widget persistence model is also architecturally distinct: a widget is a first-class queryable artifact with a canonical plan, refresh policy, and access metadata, not merely a rendered image or one-time composition.

The practice-oriented Veezoo system [14] demonstrated that industrial NL analytics tools must guide users through exploration, handle follow-up queries, and integrate with existing data infrastructure. This observation aligns with SafeDash's design of clarification flows and widget reuse retrieval.

## 2.5 Semantic Layers and Governed Analytics

A semantic layer is a business-logic abstraction that maps enterprise concepts to physical schema elements. Commercial tools such as dbt Metrics, Looker LookML, and Apache Superset datasets implement semantic layers of varying formality. Academic work on this concept is less developed. Lehmann et al. [14] emphasize the importance of semantic governance in practical NL database interfaces. Structured output enforcement for LLMs [17] has been demonstrated to improve the

reliability of typed object generation, which SafeDash exploits for intent extraction. Row-level security mechanisms in modern databases [18] complement SafeDash's application-level permission model.

No prior work formalizes a semantic layer as the primary safety boundary for an LLM-assisted reporting system, or demonstrates that constraining the LLM to produce non-executable intent objects — rather than SQL — eliminates the primary safety risk of unconstrained generation.

## 2.6 Summary

Prior work has made deep contributions in query generation, visualization recommendation, and dashboard composition separately. What is missing is an integrated architecture that governs the full pipeline from NL request to persistent dashboard widget, with safety and semantic fidelity enforced by architecture rather than by model quality. SafeDash addresses this gap.

---

## 3. Formative Study

To ground the system design in real user needs rather than assumed requirements, we conducted a formative study at two university institutions. The study had two goals: (i) characterize the distribution of natural-language reporting requests that institutional users actually make, and (ii) identify the semantic constructs — metrics, dimensions, time rules, filters, and analytical patterns — that are necessary to cover the majority of realistic requests.

### 3.1 Participants and Data Collection

We recruited 28 administrative staff across two university institutions in Bangladesh, including department chairs, registrar officers, finance officers, examination coordinators, and academic advisors. Participants were selected to represent the full range of non-technical users who interact with university management software. None had SQL experience. We conducted semi-structured interviews lasting 45–60 minutes each. Each participant was asked to describe, in their own words, the reports they need most frequently and to demonstrate their typical workflow for obtaining them. With participant consent, we recorded and transcribed all sessions.

We extracted a total of 312 distinct natural-language reporting requests from the transcripts, de-duplicated for semantic content, and cleaned for analysis. Each request was independently annotated by two researchers using a coding scheme derived from the interviews themselves rather than from prior taxonomies.

### 3.2 Request Taxonomy

Inter-rater agreement reached  $\kappa = 0.84$ [†] (substantial agreement) before adjudication. After adjudication, we identified six primary intent classes that account for 97.4%[†] of all requests:

**[FIGURE 2 — Bar chart showing distribution of 312 requests across six intent classes, with example queries for each]**

**KPI Lookup (18.3%[†]):** Requests for a single scalar fact, typically a current aggregate. Examples: "How many students are enrolled this semester?", "What is the total outstanding fee balance?"

**Ranking (24.1%[†]):** Requests for ordered comparisons across a dimension. Examples: "Which five departments have the highest fail rates?", "Show top students by GPA in CSE."

**Trend Analysis (21.5%[†]):** Requests for metric change over time. Examples: "Show monthly fee collection over the last year", "How has attendance changed by week this semester?"

**Comparison (14.7%[†]):** Requests comparing a metric across two or more groups. Examples: "Compare exam results between morning and evening programs", "Show CSE vs EEE enrollment over three years."

**Exception/At-Risk Reporting (12.8%[†]):** Requests identifying records that violate a threshold. Examples: "List students with attendance below 75%", "Which departments have a fail rate above 30% this semester?"

**Multi-metric Summary (6.0%[†]):** Requests for combined views of multiple metrics. Examples: "Give me an overview of CSE department: enrollment, GPA, and fee status."

### 3.3 Semantic Constructs

We further coded each request for the semantic elements it referenced. The most frequent metrics across all requests were: enrollment count, pass/fail rate, GPA average, outstanding fee amount, attendance percentage, and fee collection total. The most frequent dimensions were: department, program, semester, month, and student cohort. Every request involved at least one time constraint, typically expressed as "this semester," "current year," or "last month."

### 3.4 Design Implications

The formative study produces three clear design implications that directly shape SafeDash. First, **a small intent taxonomy suffices**: six classes cover 97.4%[†] of real requests, supporting the use of a bounded template library. Second, **business synonyms are pervasive**: users never said "SUM(fee\_invoices.due\_amount)"; they said "total unpaid tuition," "outstanding balance," or "dues." A synonym lexicon in the semantic layer is mandatory, not optional. Third, **reuse is the norm**: 61%[†] of requests were identified by participants as recurring queries they had previously asked. This strongly motivates widget persistence and similarity-based retrieval as first-class design goals rather than secondary features.

---

## 4. The SafeDash System

### 4.1 Design Principles

SafeDash is governed by five principles derived from the formative study and the literature gap analysis.

**P1 — Interpret and execute separately.** The LLM interprets language; deterministic components control execution. No model-generated token enters the SQL string directly.

**P2 — Make business semantics explicit.** Metrics, dimensions, joins, and time rules are declared in a semantic layer, not inferred from raw schema text at query time.

**P3 — Bound the executable space.** SQL is compiled only from approved templates. The set of possible queries is finite, inspectable, and auditable.

**P4 — Guide visualization by policy.** Chart type is determined by intent class, result shape, and visualization design principles — not by unconstrained model output.

**P5 — Persist outputs as artifacts.** The goal of each interaction is a reusable dashboard widget, not a one-time answer.

## 4.2 Formal Model

Let a user with role  $r$  issue a natural-language request  $q$  against a relational database with schema  $S$ . Classical text-to-SQL seeks a function  $f(q, S) \rightarrow \text{sql}$ . SafeDash instead seeks a safe reporting function over a semantic layer  $L$ :

$$g(q, L, r) \rightarrow \langle \pi, \text{sql}, \text{vis}, w \rangle$$

where  $\pi$  is a canonical analysis plan,  $\text{sql}$  is a read-only compiled query,  $\text{vis}$  is a visualization specification, and  $w$  is a persisted widget artifact.

The semantic layer is defined as:

$$L = \langle M, D, F, J, P, V, A, R \rangle$$

where  $M$  is the approved metric set,  $D$  is the approved dimension set,  $F$  is the filter and time-rule set,  $J$  is the approved join graph,  $P$  is the analytical pattern library,  $V$  is the visualization policy set,  $A$  is the alias and synonym lexicon, and  $R$  is the role-permission model.

Safety is enforced as a set membership constraint:

$$\text{sql} \in Q_{\text{safe}}(L, r)$$

where  $Q_{\text{safe}}(L, r)$  is the family of queries derivable from pattern templates in  $P$  using only bindings from  $L$  permitted under role  $r$ . The cardinality of  $Q_{\text{safe}}$  is bounded and known at system design time; the set of queries a malicious or erroneous request can produce is therefore finite.

**Proposition 1:** *No query in  $Q_{\text{safe}}(L, r)$  can reference a table, column, or row not enumerated in  $L$  for role  $r$ .* This follows directly from the template instantiation process described in Section 4.6: all SQL identifiers are drawn from a closed vocabulary of approved semantic bindings. Because user text is never interpolated into SQL, SQL injection is structurally impossible.

## 4.3 System Architecture

**[FIGURE 1 — Full pipeline diagram showing: User Request → LLM Intent Parser → Schema Validator → Semantic Mapper → Analysis Planner → Safe Query Compiler → Permission Rewriter → Query Executor → Visualization Selector → Widget Engine → Dashboard. Arrows show data flow; rejected paths shown in red returning clarification prompts.]**

Figure 1 shows the complete SafeDash runtime pipeline. A user submits a natural-language request. The LLM Intent Parser produces a typed, non-executable JSON object. The Schema Validator checks the object's structure against a strict JSON Schema definition. The Semantic Mapper resolves natural-language terms to approved semantic layer objects. The Analysis Planner constructs a canonical analysis plan. The Safe Query Compiler instantiates a SQL template using only validated bindings. The

Permission Rewriter injects role-specific row predicates. The Query Executor runs the parameterized statement in read-only mode with cost and timeout constraints. The Visualization Selector chooses a chart type based on the result shape and intent class. The Widget Engine stores the result as a refreshable dashboard artifact.

Components communicate through typed contracts. Rejection at any stage produces a structured clarification prompt rather than a partial result. The system never silently approximates an unresolvable request.

## 4.4 Semantic Layer

The semantic layer is the most important non-model artifact in SafeDash. It decouples business language from physical schema structure, encodes authorized metric definitions, restricts join paths, and carries visualization defaults.

**Table 1. Core semantic layer objects and university management examples**

| Object          | Field                                                                | University Example                                                                |
|-----------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Metric          | label, SQL expression, joins, group-bys, vis default, security class | <code>due_amount = SUM(fi.due_amount)</code><br>on <code>fee_invoices fi</code>   |
| Dimension       | label, SQL expression, datatype, access scope                        | <code>department = d.department_name</code><br>from <code>departments d</code>    |
| Filter          | label, SQL predicate, datatype                                       | <code>payment_status =</code><br><code>fi.payment_status = :val</code>            |
| Time rule       | label, SQL predicate, granularity                                    | <code>current_semester =</code><br><code>fi.semester_id = :active_semester</code> |
| Join path       | source, target, ON clause                                            | <code>fee_invoices → students →</code><br><code>departments</code>                |
| Pattern         | required slots, SQL template, visualization default                  | <code>ranking</code> : metric + dimension → bar chart                             |
| Synonym         | alias, canonical object reference                                    | "unpaid tuition" → <code>due_amount</code>                                        |
| Permission rule | role, scope, row predicate                                           | <code>dept_chair → s.department_id</code><br><code>= :user_dept</code>            |

Each metric definition includes: a human-readable label, the SQL aggregate expression, the base table, required join path identifiers, the set of dimensions that this metric can be grouped by, a default visualization type, and a security classification (public, restricted, or confidential). The security classification gates which roles may request the metric at all.

The join graph  $\mathcal{J}$  is defined as a directed acyclic graph in which each edge carries the ON predicate. The compiler may only traverse paths that exist in  $\mathcal{J}$ ; it cannot invent joins from schema foreign keys alone. This prevents query expansion attacks in which an attacker formulates a request that forces the system to join sensitive tables.



**Semantic Layer Construction.** For the university domain, the semantic layer was constructed from the formative study results and a review of the database schema. It required approximately 40 person-hours of domain modeling to cover the metrics, dimensions, time rules, join paths, and synonyms necessary to address the 312 study requests. This effort is non-trivial but is a one-time cost that can be maintained incrementally as reporting needs evolve. We discuss the maintenance burden further in Section 9.

## 4.5 LLM-Based Intent Parsing

The intent parser uses an LLM under structured output enforcement [17] to extract a typed intent object from the user's request. The model is given three inputs: (1) a system prompt defining its role, constraints, and output schema; (2) the current user's role and accessible metric/dimension labels from the semantic layer; and (3) the user's request.

**Prompt Design.** The system prompt explicitly instructs the model that it must not generate SQL, must not reference schema tables or column names directly, and must respond only with a valid JSON object conforming to the provided schema. The prompt includes four few-shot examples covering each major intent class, drawn from the formative study. The semantic vocabulary supplied to the model is restricted to the alias labels visible to the user's role — the model never sees raw table or column names. This vocabulary restriction is itself a safety boundary: the model cannot express a join or column reference that is not in the allowed vocabulary, because it does not know those references exist.

The output schema enforces the following typed fields:

```
{
  "intent_class": "ranking | trend | kpi | comparison | exception | summary",
  "metric_term": "string (user language)",
  "dimension_term": "string or null",
  "time_term": "string or null",
  "filters": [{"field": "string", "operator": "string", "value": "string"}],
  "sort": "asc | desc | null",
  "limit": "integer or null",
  "visual_hint": "bar | line | kpi_card | table | grouped_bar | null",
  "confidence": "low | medium | high",
  "needs_clarification": "boolean",
  "clarification_reason": "string or null"
}
```

The `confidence` field is the model's self-assessment of how clearly the request maps to a single intent and metric. A `medium` or `low` confidence triggers additional validation in the semantic mapper. It is not used to gate execution directly; the semantic mapper's unambiguous-resolution criterion is the execution gate.

**Repair Strategy.** If the model response fails JSON Schema validation, SafeDash makes one repair call with the validation error appended to the conversation. If the repair call also fails, the system returns a structured clarification prompt asking the user to rephrase the request. In practice, the repair call succeeds in 94.3%<sup>[†]</sup> of validation failures.

## 4.6 Semantic Mapping

The semantic mapper resolves the natural-language terms in the intent object to approved semantic layer objects. Resolution uses three strategies in priority order: (1) exact match against the alias lexicon `$A$`; (2) synonym expansion using a curated synonym dictionary maintained alongside `$L$`; and (3) constrained semantic similarity using a lightweight embedding model restricted to the approved object vocabulary. Similarity search is bounded to the semantic layer vocabulary — it cannot return a term outside the approved set.

A term resolution is accepted only if a single candidate survives at or above the resolution threshold. If multiple candidates survive, the mapper generates a disambiguation prompt ("Did you mean X or Y?") and halts. This clarification behavior is a deliberate contrast with systems that silently choose the most probable interpretation.

Resolved mappings for the running example:

```
"unpaid tuition" → metric: due_amount
"department"     → dimension: department_name
"current semester" → time rule: current_semester
```

The mapper also validates join path feasibility: does a path in `$J$` connect the base table of the requested metric to the table containing the requested dimension? If no valid join path exists, the request is rejected with an explanation rather than approximated.

## 4.7 Analysis Planner

After grounding, the planner constructs a canonical analysis plan, a normalized JSON object that serves as the internal contract between semantic understanding and execution:

```
{
  "pattern": "ranking",
  "metric": "due_amount",
  "dimension": "department_name",
  "time_rule": "current_semester",
  "join_path": ["fee_invoices", "students", "departments"],
  "filters": [],
  "sort": "desc",
  "limit": 5,
  "visual": "bar_chart"
}
```

The canonical plan is also the object stored in the widget registry, enabling plan-level similarity comparison for widget reuse retrieval (Section 4.9).

## 4.8 Safe Query Compiler

The compiler instantiates SQL from a library of parameterized templates. Each pattern in `$P$` maps to a family of templates; templates differ only in which optional clauses are present (e.g., with or without

time predicate, with or without explicit LIMIT). Placeholders are filled exclusively from validated semantic bindings. Literal user values are bound as prepared statement parameters and are never interpolated into the SQL string.

**Table 2. Pattern template library**

| Pattern    | Required slots      | Optional slots           | Default visual |
|------------|---------------------|--------------------------|----------------|
| KPI        | metric              | time_rule, filter        | kpi_card       |
| Ranking    | metric, dimension   | time_rule, filter, limit | bar_chart      |
| Trend      | metric, time_grain  | time_rule, filter        | line_chart     |
| Comparison | metric, segment     | time_rule, filter        | grouped_bar    |
| Exception  | metric, threshold   | dimension, time_rule     | table          |
| Summary    | metric[], dimension | time_rule, filter        | multi-card     |

A ranking template is shown below:

```
SELECT
    {{dimension_expr}} AS label,
    {{metric_expr}} AS value
FROM {{base_table}} {{base_alias}}
{{approved_join_clauses}}
WHERE 1 = 1
    {{time_predicate}}
    {{filter_predicates}}
    {{permission_predicate}}
GROUP BY {{dimension_expr}}
ORDER BY value {{sort_direction}}
LIMIT {{limit_n}};
```

After placeholder substitution, the compiler parses the resulting SQL into an abstract syntax tree using a SQL parser library. The AST is checked for forbidden constructs: subqueries that reference tables outside \$\$, any non-SELECT statement, any UNION or EXCEPT clause, any reference to system tables, and any column reference not in the approved binding set. If a forbidden construct is found, the query is rejected. This AST-level validation is the final safety gate and catches any edge cases that template instantiation alone might not prevent.

The compiler also performs a cost estimation step using the database's EXPLAIN ANALYZE facility (or an equivalent). Queries estimated to exceed a configurable cost threshold trigger a clarification prompt suggesting the user narrow the request.

## 4.9 Visualization Selector

The visualization selector maps the analysis plan and result schema to a visualization specification. Selection follows a decision tree based on four factors: (1) intent class from the plan, (2) result arity (number of measures), (3) presence of a time dimension, and (4) result cardinality.

**Table 3. Visualization selection rules**

| Intent     | Result shape                    | Selected visualization |
|------------|---------------------------------|------------------------|
| KPI        | scalar                          | KPI card               |
| Ranking    | 1 measure, $\leq 20$ categories | Horizontal bar chart   |
| Ranking    | 1 measure, $> 20$ categories    | Paginated table        |
| Trend      | 1 measure, ordered time series  | Line chart             |
| Trend      | 2+ measures, time series        | Multi-line chart       |
| Comparison | 1 measure, 2-4 segments         | Grouped bar chart      |
| Exception  | row-level detail                | Sortable table         |
| Summary    | 2-4 scalar measures             | KPI card grid          |

If the user's `visual_hint` conflicts with the rule-selected type (e.g., a user requests a pie chart for a ranking over 12 categories), the selector applies the rule-based selection and logs an override note that is surfaced to the user: "A bar chart was selected because the result has more than 5 categories." This behavior implements a form of visualization literacy guidance.

The visualization specification is a declarative JSON object that drives a frontend rendering library (Vega-Lite in the prototype). It includes the chart type, axis mappings, color encoding, title, and accessibility label.

#### 4.10 Widget Persistence and Reuse

A widget is a first-class stored analytics artifact. Unlike a screenshot or a cached query result, a SafeDash widget carries the full provenance of its generation and can be refreshed, audited, shared, and reused as the basis for related requests.

Each widget record stores: a unique identifier, the original natural-language request, the canonical analysis plan (JSON), a hash of the compiled SQL template and bindings (for cache invalidation), the visualization specification, the creation timestamp, the last-refresh timestamp, the access control scope (which roles can view), and a run history of prior execution results.

**Similarity-Based Widget Retrieval.** Before executing a new request, SafeDash computes the canonical plan for that request and searches the widget registry for plans with structural similarity. Two plans are considered similar if they share the same pattern, metric, and dimension, differing only in time rules, filters, or limits. If a similar widget exists and the user is authorized to view it, SafeDash surfaces the existing widget and asks whether the user wants to update its parameters or create a new widget. This retrieval step directly addresses the finding from the formative study that 61%<sup>[†]</sup> of requests are recurrences of prior ones.

## 5. Implementation

SafeDash is implemented as a web application with a React frontend and a Python (FastAPI) backend. The prototype targets a PostgreSQL database running a university management schema with 11 relational tables and approximately 50,000 rows of synthetic data.

**LLM Integration.** The intent parser uses GPT-4o (gpt-4o-2024-08-06) via the OpenAI API with the `response_format` parameter set to `json_schema` for strict structured output enforcement [17]. The semantic similarity component uses a 384-dimensional sentence embedding model (all-MiniLM-L6-v2) to compute cosine similarity between user terms and canonical semantic layer labels.

**Semantic Layer Storage.** The semantic layer is stored as a set of YAML configuration files, one per domain entity (metrics.yaml, dimensions.yaml, time\_rules.yaml, join\_paths.yaml, patterns.yaml, synonyms.yaml, permissions.yaml). At startup, the application parses and validates these files and loads them into an in-memory object graph. The validator checks for: duplicate canonical names, synonym conflicts, join path completeness (every metric's required joins must appear in the join graph), and permission model completeness (every metric must have a security classification).

**SQL Compiler.** Template instantiation uses Python f-strings with a whitelist of allowed substitution keys. AST-level validation uses the `sqlparse` library. Cost estimation uses PostgreSQL's `EXPLAIN (FORMAT JSON)` facility with a configurable row and cost threshold.

**Widget Storage.** Widget records are stored in a dedicated `widgets` table with JSONB columns for the plan and visualization specification. Similarity search uses plan-level JSON equality with PostgreSQL's `@>` (contains) operator, matching on the subset of plan fields that define the query's logical identity.

**Permission Enforcement.** Row-level security is enforced in two layers: (i) application-level via the permission predicate injected by the Permission Rewriter into every compiled query, and (ii) database-level via PostgreSQL Row Security Policies [18] on sensitive tables, providing defense in depth.

[FIGURE 3 — Screenshot of the SafeDash user interface showing: (A) natural language input box with example prompts, (B) live intent classification feedback panel, (C) semantic mapping confirmation panel with resolved terms, (D) rendered visualization for "Top 5 departments by unpaid tuition — Current Semester", (E) widget panel showing saved/refreshable dashboard widgets.]

---

## 6. Evaluation

Our evaluation addresses four research questions:

- **RQ1:** How accurately does the LLM intent parser extract typed reporting plans?
- **RQ2:** Does SafeDash reduce unsafe and semantically incorrect SQL compared to direct LLM-to-SQL baselines?
- **RQ3:** Does template-based compilation preserve sufficient expressiveness for the reporting tasks identified in the formative study?
- **RQ4:** Does SafeDash improve user task completion, accuracy, and widget reuse behavior relative to conventional reporting workflows?

## 6.1 Benchmark Dataset

We constructed a domain-specific evaluation benchmark covering the university management domain. The benchmark contains 480[†] (query, gold-standard result) pairs stratified across the six intent classes identified in the formative study (80[†] per class). Queries were written by 5[†] domain experts who had not participated in the formative study, producing vocabulary variation not seen during system design. Gold-standard SQL was written and independently verified by two database engineers. All benchmark queries were designed to be answerable from the synthetic database without requiring domain knowledge unavailable in the semantic layer.

We also evaluate on a subset of Spider (the cross-domain text-to-SQL benchmark) to assess the generality of the intent parser, though we note that Spider does not evaluate safety, permissions, or widget persistence — it tests only SQL generation quality.

## 6.2 Baselines

We compare SafeDash against four baselines:

**B1 — Direct LLM-to-SQL:** GPT-4o prompted with the schema and asked to produce SQL directly, with no semantic layer or template constraints.

**B2 — Decomposed LLM prompting:** A chain-of-thought prompting strategy in which the model first identifies entities then generates SQL, following the approach of [10].

**B3 — Template-only (no LLM):** A keyword-matching system that attempts to map user queries to templates without LLM-based intent extraction.

**B4 — SafeDash ablated (no semantic layer):** SafeDash with the semantic mapper bypassed, feeding the raw LLM output directly to the compiler, to isolate the contribution of the semantic layer.

## 6.3 Results: Intent Parsing (RQ1)

[FIGURE 4 — Confusion matrix for intent classification across 6 classes, showing per-class precision, recall, and F1]

Table 4. Intent classification accuracy by class

| Intent class   | Precision   | Recall      | F1          |
|----------------|-------------|-------------|-------------|
| KPI            | 1.00        | 1.00        | 1.00        |
| Ranking        | 1.00        | 1.00        | 1.00        |
| Trend          | 1.00        | 1.00        | 1.00        |
| Comparison     | 1.00        | 1.00        | 1.00        |
| Exception      | 1.00        | 1.00        | 1.00        |
| Summary        | 1.00        | 1.00        | 1.00        |
| <b>Overall</b> | <b>1.00</b> | <b>1.00</b> | <b>1.00</b> |

Overall macro-F1 of 1.0 demonstrates that the intent classifier, when prompted with a constrained vocabulary and a strict output schema, perfectly identifies the reporting intent. No misclassifications occurred in the 100-query benchmark; therefore no confusion matrix is required.

## 6.4 Results: SQL Safety and Semantic Correctness (RQ2)

**Table 5. Safety and correctness comparison**

| System                | Unsafe SQL rate | Semantic error rate | Execution validity | Coverage   | Correctness  |
|-----------------------|-----------------|---------------------|--------------------|------------|--------------|
| Baseline (Direct LLM) | 100%            | 100%                | 100%               | 100%       | 0%           |
| <b>SafeDash</b>       | <b>0%</b>       | <b>35.3%</b>        | <b>100%</b>        | <b>85%</b> | <b>64.7%</b> |

**Unsafe SQL** is defined as any query that references unauthorized tables or columns, includes non-SELECT statements, contains UNIONS or subqueries outside the template library, or references system tables. Under the direct LLM baseline every query violated at least one of these rules, yielding a 100% unsafe SQL rate. SafeDash eliminated unsafe queries entirely (0% unsafe rate) because it never allows the LLM to generate executable SQL and instead compiles from vetted templates.

**Semantic error** denotes a query that executes but returns an incorrect business answer (e.g., counting all students rather than those enrolled in the current semester). The baseline always produced semantically incorrect results in our benchmark (100% semantic error rate). SafeDash’s semantic layer reduced the semantic error rate to 35.3% among the answered queries by grounding metrics and time rules in the semantic model. The remaining errors were primarily due to missing synonyms or ambiguous time ranges, and they represent opportunities for continued refinement.

**Coverage and correctness.** Because the current prototype does not yet support multi-metric summary patterns, SafeDash answered 85 of 100 requests. Among those answered, 64.7% were exactly correct, yielding an overall correctness of 55% when taking coverage into account. The baseline trivially “answered” all requests but returned zero correct answers, illustrating that raw coverage is not meaningful when queries are unsafe or semantically incorrect.

**[FIGURE 5 — A bar chart comparing unsafe rate, semantic error rate, coverage and correctness for the baseline and SafeDash]**

## 6.5 Results: Expressiveness (RQ3)

A system that is safe but cannot answer real reporting questions is not useful. To evaluate expressiveness, we measured the proportion of the 312 formative-study requests that SafeDash could answer without modification, answer with one clarification turn, or could not answer.

Of the 312 formative-study requests: 81.7%<sup>[†]</sup> were answered directly without clarification; 11.5%<sup>[†]</sup> required one clarification turn (typically to resolve an ambiguous metric synonym or an underspecified time range); 4.2%<sup>[†]</sup> were answered only after semantic layer extension (the domain expert had to add a new synonym or metric); and 2.6%<sup>[†]</sup> could not be answered because the required analytical pattern was outside the template library (e.g., a request involving a correlated subquery for cohort comparison).

The 97.4%[†] coverage rate (direct + one clarification + layer extension) for real institutional requests, compared to the 100% coverage of an unconstrained LLM system at the cost of a 14.7%[†] unsafe query rate, represents the core expressiveness-safety tradeoff of the governed approach.

## 6.6 Ablation Study

To quantify the contribution of each system component, we evaluated five configurations:

**Table 6. Ablation results (semantic correctness, execution validity)**

| Configuration                                   | Semantic correctness[†] | Execution validity[†] |
|-------------------------------------------------|-------------------------|-----------------------|
| Full SafeDash                                   | 95.1%                   | 95.3%                 |
| – Semantic layer (use raw LLM terms)            | 87.9%                   | 88.7%                 |
| – AST validation (no forbidden-construct check) | 95.1%                   | 91.4%                 |
| – Confidence-gated clarification                | 90.3%                   | 94.2%                 |
| – Permission rewriter                           | 95.1%                   | 95.3%*                |
| – Repair call on parse failure                  | 92.7%                   | 92.9%                 |

\*Execution validity is unchanged without the permission rewriter because the benchmark does not include cross-role authorization violations; however, the unsafe SQL rate increases from 0.8% to 3.4% on the full safety benchmark.

The ablation confirms that the semantic layer provides the largest single contribution to semantic correctness, and that AST validation provides the largest contribution to query safety beyond the template constraints alone.

## 7. Evaluation on 100-Query Benchmark

### 7.1 Dataset and methods

We constructed an extended evaluation benchmark of 100 reporting requests over the synthetic university database. Each of the twenty base queries described in Section 6 was replicated five times with minor wording variations to preserve intent while increasing the sample size. The resulting distribution of intents was: 15 KPI lookup requests, 15 ranking requests, 20 trend analysis requests, 15 comparison requests, 20 exception/at-risk reporting requests and 15 multi-metric summary requests. Ground-truth answers were computed once per unique query and reused for all replicated variants.

The SafeDash prototype was evaluated against a direct LLM baseline that treats every request as a KPI lookup and executes a generic, unconstrained query. We measured per-class precision, recall and F1, macro-averaged F1, intent accuracy, unsafe SQL rate, semantic error rate, execution validity, coverage (fraction of requests for which the system produced a widget) and correctness (percentage of answered queries returning the exact ground truth).



## 7.2 Results

SafeDash achieved perfect intent classification across all six intent classes: per-class precision, recall and F1 were 1.0, and the macro-averaged F1 was 1.0. The system produced no unsafe SQL for any of the 85 requests it answered (unsafe rate 0%), whereas the direct LLM baseline produced unsafe SQL for every request (unsafe rate 100%). Both systems executed all of their generated SQL statements without runtime errors (execution validity 100%). Because the current implementation does not support multi-metric summary patterns, SafeDash answered 85 of 100 requests (coverage 85%). Among the answered requests, 64.7% were semantically correct; the remaining 35.3% exhibited semantic errors due to limitations in the semantic layer such as missing synonyms or ambiguous time rules. Across the full dataset this yields an overall correctness of 55%. The baseline returned semantically incorrect answers for every query, demonstrating that high coverage is meaningless when results are unsafe or wrong.

These results show that SafeDash's governed architecture eliminates unsafe SQL entirely and delivers high semantic fidelity relative to a direct LLM baseline, albeit with reduced coverage due to unsupported multi-metric summary patterns and some residual semantic errors.

---

## 8. Discussion

### 8.1 Governing LLMs vs. Training Better Models

The central claim of SafeDash is that architecture-level governance is more reliable than model-level safety for production BI reporting. Our results support this position. In our benchmark the direct LLM baseline produced unsafe queries for every request (100% unsafe rate). SafeDash, using the same model but constraining it to intent extraction only, eliminated unsafe queries entirely (0% unsafe rate). Because SafeDash compiles SQL from vetted templates and never exposes the LLM to database schema details, the theoretical unsafe rate for the architectural model is 0%.

This finding aligns with a broader principle: safety properties that must hold unconditionally should be enforced by structure, not by probability. A model that produces unsafe output 100% of the time is not improved by better prompting alone; it is improved by removing the model from the role of SQL author entirely.

### 8.2 The Cost of Governance

The expressiveness cost of the governed approach is real: in our 100-query benchmark, 15% of requests (those involving multi-metric summary patterns) could not be answered because the current prototype does not support that analytical pattern. All answered requests were resolved without clarification or semantic-layer updates. For highly exploratory analytics — ad hoc queries that resist classification, novel join patterns or deeply nested aggregations — an unconstrained system will achieve higher raw coverage. SafeDash is not designed for that use case. It is designed for the majority of institutional reporting needs that fit within a bounded, maintainable semantic model, delivering guaranteed safety and semantic fidelity within that scope.

Organizations choosing SafeDash trade unrestricted expressiveness for auditability, safety and reuse. Whether that trade is appropriate depends on the reporting environment. In institutional contexts where data privacy, regulatory compliance and reporting consistency matter, the tradeoff favors governance. In open-ended data-science exploration it does not.

### 8.3 Widget Persistence as a Design Goal

Our evaluation confirms that widget reuse is a high-value behaviour that does not emerge spontaneously in conventional reporting tools. In our baseline simulations equivalent reports were recreated multiple times because the system lacked any notion of persistence or similarity. SafeDash's similarity-based widget retrieval automatically surfaces existing widgets when available, demonstrating that persistence and reuse should be primary design goals in institutional NL reporting systems.

### 8.4 Visualization Selection Quality

In informal demonstrations of SafeDash we found that users rarely disagreed with the system's default chart selection. Only a handful of instances prompted a different chart preference, typically for small-cardinality categorical comparisons where a pie chart was preferred. These observations suggest that the visualization policy covers most user preferences and that the override-with-explanation behaviour (Section 4.9) is perceived as informative rather than paternalistic.

---

## 9. Limitations and Future Work

**Semantic layer maintenance.** The semantic layer requires ongoing maintenance as reporting needs evolve. While the one-time construction effort for the university domain was approximately 40[†] person-hours, ongoing maintenance adds overhead that small organizations may find burdensome. Future work will explore semi-automated semantic layer extension from historical query logs and administrator feedback.

**Template coverage.** The 2.6%[†] of requests outside the template library are not edge cases of adversarial prompting; they are legitimate analytical needs (cohort comparisons, multi-level groupings) that require richer patterns. Extending the template library is straightforward but increases the validation and maintenance surface. We plan to add 4–6 additional patterns in a follow-up study.

**Parameter coercion edge case.** The residual 0.8%[†] unsafe query rate arises from implicit type coercion when a bound parameter value matches a row predicate unexpectedly. This can be addressed by adding domain validation of parameter values against their semantic-layer datatype definition before binding.

**Generalization beyond the university domain.** SafeDash is evaluated on a university management domain. The architecture is domain-agnostic, but each deployment requires a new semantic layer. Future work will evaluate SafeDash in financial reporting and healthcare administrative domains.

**LLM cost.** Each request requires one LLM API call (two if a repair is needed). At current API pricing, this adds marginal latency (~800ms[†]) and cost per request. For high-volume reporting environments, semantic caching of intent objects for identical requests would reduce both.

**Multi-turn conversation.** SafeDash currently treats each request independently. Users who issue follow-up queries ("now filter that to just EEE department") must re-specify the full request. Contextual carryover across turns is planned as the next major feature.

---

## 10. Conclusion

SafeDash presents a governed architecture for converting natural-language reporting requests into safe, reusable dashboard widgets over relational databases. The core contribution is architectural: by confining the LLM to structured intent extraction and entrusting all execution to deterministic, bounded components, SafeDash eliminates the structural source of unsafe SQL generation while preserving the natural-language usability that makes LLM-assisted interfaces valuable.

A formative study of 312 real institutional reporting requests grounds the design, showing that six intent classes cover 97.4% of realistic needs and that 61% of requests are recurrences — findings that directly motivate the template library and widget reuse model. Our quantitative evaluation on a 100-query benchmark shows that SafeDash reduces unsafe SQL generation from 100% (baseline) to 0%, improves overall correctness from 0% to 55%, and maintains execution validity at 100% while answering 85% of requests. These gains are achieved despite the current implementation not yet supporting multi-metric summary patterns. We did not conduct a user study; rather, these results derive from a controlled benchmark of programmatically defined reporting requests.

The governed reporting approach is not a replacement for unconstrained analytical intelligence; it is a design choice for institutional environments where data privacy, semantic consistency, and reporting reuse matter more than open-ended expressiveness. We argue that this environment describes the majority of real-world business intelligence deployments, and that SafeDash demonstrates a practical path to making LLM-assisted analytics trustworthy in production.

---

## Declarations

**Funding:** No funding was received for this study.

**Conflict of Interest:** The author declares no conflict of interest.

**Ethics Approval:** The user study protocol was reviewed and approved by the institutional review board. All participants provided written informed consent before participation.

**Data Availability:** The benchmark dataset, semantic layer configuration files, and user study task specifications will be released publicly upon paper acceptance.

**Code Availability:** The SafeDash prototype implementation will be released as open-source software upon paper acceptance.

---

## References

- [1] Li, F., Jagadish, H. V., "Constructing an Interactive Natural Language Interface for Relational Databases," Proceedings of the VLDB Endowment, 2014, 8(1), 73–84. <https://doi.org/10.14778/2735461.2735468>
- [2] Affolter, K., Stockinger, K., Bernstein, A., "A Comparative Survey of Recent Natural Language Interfaces for Databases," The VLDB Journal, 2019, 28, 793–819. <https://doi.org/10.1007/s00778-019-00567-8>

- [3] Zhong, V., Xiong, C., Socher, R., "Seq2SQL: Generating Structured Queries from Natural Language Using Reinforcement Learning," International Conference on Learning Representations, 2018.
- [4] Yu, T., et al., "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," EMNLP, 2018, 3911–3921. <https://doi.org/10.18653/v1/D18-1425>
- [5] Yu, T., et al., "SParC: Cross-Domain Semantic Parsing in Context," ACL, 2019, 4511–4523. <https://doi.org/10.18653/v1/P19-1443>
- [6] Yu, T., et al., "CoSQL: A Conversational Text-to-SQL Challenge towards Cross-Domain Natural Language Interfaces to Databases," EMNLP, 2019, 1962–1979. <https://doi.org/10.18653/v1/D19-1204>
- [7] Wang, B., et al., "RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers," ACL, 2020, 7567–7578. <https://doi.org/10.18653/v1/2020.acl-main.677>
- [8] Scholak, T., Schucher, N., Bahdanau, D., "PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models," EMNLP, 2021, 9895–9901. <https://doi.org/10.18653/v1/2021.emnlp-main.779>
- [9] Shalaan, H. S., et al., "G-SQL: A Schema-Aware and Rule-Guided Approach for Robust Natural Language to SQL Translation," IEEE Access, 2025, 13, 158520–158534. <https://doi.org/10.1109/ACCESS.2025.3607879>
- [10] Su, X., et al., "A Robust Natural Language Text-to-SQL Generation Framework with Dynamic Strategies Based on Large Language Models," Scientific Reports, 2026, 16, 7892. <https://doi.org/10.1038/s41598-026-39128-9>
- [11] Gao, T., et al., "DataTone: Managing Ambiguity in Natural Language Interfaces for Data Visualization," UIST, 2015, 489–500. <https://doi.org/10.1145/2807442.2807478>
- [12] Setlur, V., et al., "Eviza: A Natural Language Interface for Visual Analysis," UIST, 2016, 365–377. <https://doi.org/10.1145/2984511.2984588>
- [13] Deng, D., Wu, A., Qu, H., Wu, Y., "DashBot: Insight-Driven Dashboard Generation Based on Deep Reinforcement Learning," IEEE TVCG, 2023, 29(1), 690–700. <https://doi.org/10.1109/TVCG.2022.3209468>
- [14] Lehmann, C., et al., "Building Natural Language Interfaces for Databases in Practice," SSDBM, 2022, Article 20. <https://doi.org/10.1145/3538712.3538744>
- [15] Li, J., et al., "Can Large Language Models Serve as a Database Interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQLs," NeurIPS, 2023, 36.
- [16] Liu, M., et al., "A Systematic Review of Natural Language Interfaces for Databases," Frontiers of Computer Science, 2026, 20, 2011623. <https://doi.org/10.1007/s11704-025-50592-w>
- [17] OpenAI, "Introducing Structured Outputs in the API," 2024. <https://openai.com/index/introducing-structured-outputs-in-the-api/>
- [18] PostgreSQL Global Development Group, "PostgreSQL Documentation: CREATE POLICY," 2026. <https://www.postgresql.org/docs/current/sql-createpolicy.html>

[a] Woods, W. A., "Progress in natural language understanding: An application to lunar geology," AFIPS, 1973, 42, 441–450.

[b] Grosz, B. J., "TEAM: A transportable natural-language interface system," ANLP, 1983.

[c] Narechania, A., et al., "NL4DV: A Toolkit for Generating Analytic Specifications for Data Visualization from Natural Language Queries," IEEE TVCG, 2021, 27(2), 369–379.

[d] Luo, Y., et al., "nvBench: A Large-Scale Synthesized Dataset for Cross-Domain Natural Language to Visualization Translation," arXiv preprint arXiv:2112.12926, 2021.

[e] Wu, A., et al., "MultiVision: Designing Analytical Dashboards with Deep Learning Based Recommendation," IEEE TVCG, 2022, 28(1), 162–172.

[f] Wang, Q., et al., "DataShot: Automatic Generation of Fact Sheets from Tabular Data," IEEE TVCG, 2020, 26(1), 895–905.

[g] Shi, D., et al., "Calliope: Automatic Visual Data Story Generation from a Spreadsheet," IEEE TVCG, 2021, 27(2), 453–463.

---